HELIX

Cybersecurity Operations Platform

PoC Engineering Roadmap

Learn & Build Hybrid Approach

Document Type	Engineering Roadmap
Project Phase	Proof of Concept (PoC)
Document Version	2.0
Academic Year	2025 – 2026
Author	Yasseene
Classification	Academic – Confidential

Contents

Introduction

This document is your personal engineering execution blueprint, adapted to your actual project architecture: a decoupled SPA frontend, a layered PHP API (Controller → Service → Model), a Python ML module, and a Java utility module. Every file reference in this roadmap maps to a real file in your tree. Follow it sequentially, create files only when you reach them, and mark your progress as you go.

How to Use This Roadmap

[x] Not started	Mark as completed when done
[ ] In progress	Currently working on
[ ] Completed	Task finished
[!] Blocked	Needs review or unblocking

You do not move to the next phase until the current phase's **Definition of Done** is satisfied. You create files only when you need them – not before.

Architecture Reference

Before starting, internalize this flow. Every feature you build will follow it:

```
Browser (frontend/)
    |
    |  HTTP fetch() via api.js
    v
api/index.php          <- Front Controller: parses route, applies middleware, dispatches
    |
    |-- middleware/AuthMiddleware.php     <- Is the user logged in?
    |-- middleware/RoleMiddleware.php     <- Is the user allowed?
    |
    v
api/controllers/XController.php         <- Parse request, call service, return response
    |
    v
api/services/XService.php               <- All business logic lives here
    |
    v
api/models/X.php                        <- All DB queries live here (PDO)
    |
    v
MySQL (helix_db)
```

Key architectural decisions already made:

- The frontend is completely decoupled – it never touches PHP directly
- `api/index.php` is the single entry point – all routes go through it
- `api/config/Cors.php` handles CORS because frontend and API are on different paths
- `api/helpers/Response.php` standardizes all JSON responses
- `api/helpers/Validator.php` centralizes input validation
- `api/services/JavaBridge.php` is the only place that touches `shell_exec()`

Global Phase Map

```
PHASE 0 -> PHASE 1 -> PHASE 2 -> PHASE 3 -> PHASE 4 -> PHASE 5 -> PHASE 6 -> PHASE 7 -> PHASE 8
  |          |          |          |          |          |          |          |
Setup     Architecture  Auth +    Dashboard   Log         Alert       ML          AI        Scan
& Git     & DB Design   RBAC      & Routing  Ingestion   Module     Detection  Assistant    Modu
```

Dependency chain (strict):

- Phase 1 requires Phase 0
- Phase 2 requires Phase 1 (DB schema + routing foundation in place)
- Phase 3 requires Phase 2 (auth middleware working, frontend routing exists)
- Phase 4 requires Phase 3 (dashboard shell exists as UI home)
- Phase 5 requires Phase 4 (log data is the raw input for alerts + ML)
- Phase 6 requires Phase 5 (alerts are generated from scored logs)
- Phase 7 requires Phase 5 (ML needs log data to score)
- Phase 8 requires Phase 6 (AI analyzes alerts enriched by ML scores)
- Phase 9 requires Phase 2 (scanner only needs auth, independent of logs/ML)
- Phase 10 requires Phases 2–9 (all modules complete and individually tested)
- Phase 11 requires Phase 10 (containerize only a stable application)

1. PHASE 0 – Project Setup, Git Workflow & Development Environment

Phase 0: Setup & Environment

Why this comes first: Without a reproducible environment and version control, everything you build is fragile. One wrong XAMPP config, one missing `.gitignore`, and you lose hours. This phase costs 2 hours and saves 20.

Objective

Establish the complete development environment, Git discipline, Apache serving strategy, Composer setup, and Python environment – before writing a single line of application logic.

Learning Goals

- Understand Git branching strategy for solo development
- Understand how Apache serves two directories under one vhost (frontend + api)
- Understand Composer as PHP's dependency manager and autoloader
- Understand `pyproject.toml` vs `requirements.txt` (and why you're using the former)
- Understand the Front Controller pattern (`api/index.php`)

Key Theoretical Concepts

- **Git Flow (simplified):** `main` = stable releases, `dev` = integration branch, `feature/*` = active work
- **Apache vhost + Alias directive:** How to serve `frontend/` at `/` and `api/` at `/api` from one domain
- **Composer autoloading (PSR-4):** How `use App\Controllers\AuthController` resolves to a file path automatically
- **Front Controller:** One entry point that routes all requests – no 30 separate PHP files each handling their own routing

- **CORS (Cross-Origin Resource Sharing):** Why the browser blocks your `fetch('/api/...')` without the right headers

1.1 0.1 – Git Repository Setup

---

[x] Create private repository on GitHub/GitLab

[x] Clone locally into your project root

[x] Create `.gitignore`:

```
# PHP
vendor/
composer.lock (optional)

# Python
python-module/.venv/
python-module/__pycache__/
python-module/models/*.joblib

# Java
java-module/build/*.class

# Environment
api/config/.env
.env

# IDE
.vscode/
.idea/
*.DS_Store
```

[ ] Create `CHANGELOG.md` – update at end of every phase

[ ] Initial commit: `git commit -m "chore: initialize HELIX project"`

[ ] Create `dev` branch: `git checkout -b dev`

Branching strategy:

- Work on `feature/*` branches cut from `dev`
- Merge `feature/*` → `dev` when a phase is complete and tested
- Merge `dev` → `main` only when a full milestone works end-to-end

1.2 0.2 – Apache Vhost Configuration (Fedora/RHEL)

This is the critical decision from the review. You have `frontend/` and `api/` as siblings. Define how Apache serves both under `helix.local`. On Fedora, Apache config lives in `/etc/httpd/`.

Step 1: Install Apache, PHP, and dependencies

```
sudo dnf install httpd php php-cli
sudo dnf install php-pdo php-pdo_mysql php-json
sudo systemctl start httpd
sudo systemctl enable httpd
```

Verify Apache is running: `curl http://localhost`

Step 2: Create vhost configuration

Create `/etc/httpd/conf.d/helix.conf` (requires `sudo`):

```
<VirtualHost *:80>
    ServerName helix.local
    DocumentRoot /home/yasseene/HELIX/frontend

    <Directory /home/yasseene/HELIX/frontend>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted

        <IfModule mod_rewrite.c>
            RewriteEngine On
            RewriteBase /
            RewriteCond %{REQUEST_FILENAME} !-f
            RewriteCond %{REQUEST_FILENAME} !-d
            RewriteRule ^(.*)$ index.html [QSA,L]
        </IfModule>
    </Directory>

    Alias /api /home/yasseene/HELIX/api

    <Directory /home/yasseene/HELIX/api>
        Options FollowSymLinks
        AllowOverride All
        Require all granted

        <IfModule mod_rewrite.c>
            RewriteEngine On
            RewriteBase /api
            RewriteCond %{REQUEST_FILENAME} !-f
            RewriteCond %{REQUEST_FILENAME} !-d
            RewriteRule ^(.*)$ index.php?path=$1 [QSA,L]
        </IfModule>
    </Directory>

    <FilesMatch \.php$>
        SetHandler application/x-httpd-php
    </FilesMatch>

    ErrorLog /var/log/httpd/helix_error.log
    CustomLog /var/log/httpd/helix_access.log combined
</VirtualHost>
```

Step 3: Enable mod_rewrite

On Fedora/RHEL, `mod_rewrite` is usually enabled by default. Verify:

```
grep -i "rewrite" /etc/httpd/conf.modules.d/*.conf
```

Step 4: Add to system hosts file

```
echo "127.0.0.1    helix.local" | sudo tee -a /etc/hosts
ping helix.local
```

Step 5: Fix file permissions

```
sudo usermod -a -G $(id -gn) apache
chmod 755 /home/yasseene/HELIX
chmod 755 /home/yasseene/HELIX/{frontend,api}
```

Step 6: Validate and restart Apache

```
sudo httpd -t
sudo systemctl restart httpd
sudo systemctl status httpd
```

Consequence for `api.js`: All frontend API calls will use:

```
const API_BASE = "/api"; // relative -- works on same origin
```

Never hardcode `http://localhost/...` – relative paths survive domain changes.

[x] Install Apache, PHP, and dependencies

[x] Create `/etc/httpd/conf.d/helix.conf` with vhost above

[x] Add `127.0.0.1 helix.local` to `/etc/hosts`

[x] Fix permissions

[x] Verify mod_rewrite

[x] Run `sudo httpd -t` (syntax check)

[x] Run `sudo systemctl restart httpd`

[x] Verify: `http://helix.local` serves `frontend/index.html`

[x] Create `api/index.php` with `echo "API OK";` temporarily

[x] Verify: `http://helix.local/api` returns "API OK"

[x] Check logs: `tail -f /var/log/httpd/helix_error.log` if anything fails

1.3 0.3 – MySQL Setup

---

[x] Start MySQL in XAMPP

[x] Create database: `helix_db`

[x] Create dedicated user with limited privileges:

```
CREATE USER 'helix_user'@'localhost' IDENTIFIED BY 'your_password';
GRANT SELECT, INSERT, UPDATE, DELETE ON helix_db.* TO 'helix_user'@'
    localhost';
FLUSH PRIVILEGES;
```

[ ] Verify connection via phpMyAdmin

1.4 0.4 – PHP Composer Setup

`composer.json`:

```
{
  "name": "helix/api",
  "description": "HELIX Cybersecurity Platform API",
  "type": "project",
  "require": {
    "php": ">=8.1"
  },
  "require-dev": {
    "phpunit/phpunit": "^10.0"
  },
  "autoload": {
    "psr-4": {
      "App\\": "api/"
    }
  },
  "autoload-dev": {
    "psr-4": {
      "Tests\\": "tests/php/"
    }
  }
}
```

[ ] Run: `composer install`

[ ] Verify `vendor/autoload.php` was created

[ ] Add `require_once __DIR__ . '/../../vendor/autoload.php';` to `api/index.php`

Why PSR-4 matters: With this config, `use App\Controllers\AuthController` automatically maps to `api/controllers/AuthController.php`. You never write manual `require` statements for your own classes.

1.5 0.5 – Environment Configuration

`api/config/Environment.php`:

```php
<?php
namespace App\Config;

class Environment
{
    private static array $vars = [];

    public static function load(string $envFile): void
    {
        if (!file_exists($envFile)) return;

        foreach (file($envFile, FILE_IGNORE_NEW_LINES |
            FILE_SKIP_EMPTY_LINES) as $line) {
            if (str_starts_with(trim($line), '#')) continue;
            [$key, $value] = explode('=', $line, 2);
            self::$vars[trim($key)] = trim($value);
            $_ENV[trim($key)] = trim($value);
        }
    }

    public static function get(string $key, mixed $default = null): mixed
    {
        return self::$vars[$key] ?? $_ENV[$key] ?? getenv($key) ?:
            $default;
    }
}
```

Create `api/config/.env` (gitignored):

```
DB_HOST=localhost
DB_NAME=helix_db
DB_USER=helix_user
DB_PASS=your_password
OPENAI_API_KEY=sk-...
SESSION_LIFETIME=3600
PYTHON_PATH=C:/path/to/.venv/Scripts/python.exe
PYTHON_SCRIPT=python-module/anomaly_detector.py
JAVA_BUILD_PATH=java-module/build
```

Create `api/config/.env.example` (committed to git – no real values):

```
DB_HOST=localhost
DB_NAME=helix_db
DB_USER=helix_user
DB_PASS=
OPENAI_API_KEY=
SESSION_LIFETIME=3600
PYTHON_PATH=
PYTHON_SCRIPT=python-module/anomaly_detector.py
JAVA_BUILD_PATH=java-module/build
```

1.6 0.6 – Database Config

api/config/Database.php:

```php
<?php
namespace App\Config;

use PDO;
use PDOException;

class Database
{
    private static ?PDO $instance = null;

    public static function getInstance(): PDO
    {
        if (self::$instance === null) {
            $host = Environment::get('DB_HOST', 'localhost');
            $name = Environment::get('DB_NAME');
            $user = Environment::get('DB_USER');
            $pass = Environment::get('DB_PASS');

            $dsn = "mysql:host={$host};dbname={$name};charset=utf8mb4";
            $options = [
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES   => false,
            ];

            try {
                self::$instance = new PDO($dsn, $user, $pass, $options);
            } catch (PDOException $e) {
                error_log("DB connection failed: " . $e->getMessage());
                http_response_code(500);
                echo json_encode(['error' => 'Database unavailable']);
                exit;
            }
        }

        return self::$instance;
    }
}
```

[ ] Test: add a `Database::getInstance();` call to `api/index.php` temporarily → no error

1.7 0.7 – CORS Configuration

api/config/Cors.php:

```php
<?php
namespace App\Config;

```

```
class Cors
{
    public static function handle(): void
    {
        $allowedOrigins = ['http://helix.local', 'http://localhost'];
        $origin = $_SERVER['HTTP_ORIGIN'] ?? '';

        if (in_array($origin, $allowedOrigins, true)) {
            header("Access-Control-Allow-Origin: {$origin}");
        }

        header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE,
            OPTIONS');
        header('Access-Control-Allow-Headers: Content-Type, Authorization,
            X-Requested-With');
        header('Access-Control-Allow-Credentials: true');

        if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
            http_response_code(204);
            exit;
        }
    }
}
```

1.8 0.8 – Helper Classes

api/helpers/Response.php:

```
<?php
namespace App\Helpers;

class Response
{
    public static function json(mixed $data, int $status = 200): void
    {
        http_response_code($status);
        header('Content-Type: application/json');
        echo json_encode($data);
        exit;
    }

    public static function success(mixed $data = null, string $message = '
        OK', int $status = 200): void
    {
        self::json(['success' => true, 'message' => $message, 'data' =>
            $data], $status);
    }

    public static function error(string $message, int $status = 400): void
    {
        self::json(['success' => false, 'error' => $message], $status);
```

```
    }

    public static function unauthorized(): void
    {
        self::error('Unauthorized', 401);
    }

    public static function forbidden(): void
    {
        self::error('Forbidden', 403);
    }
}
```

api/helpers/Validator.php:

```
<?php
namespace App\Helpers;

class Validator
{
    private array $errors = [];

    public function required(string $field, mixed $value): self
    {
        if (empty($value) && $value !== '0') {
            $this->errors[$field] = "{$field} is required";
        }
        return $this;
    }

    public function email(string $field, mixed $value): self
    {
        if (!filter_var($value, FILTER_VALIDATE_EMAIL)) {
            $this->errors[$field] = "{$field} must be a valid email";
        }
        return $this;
    }

    public function minLength(string $field, mixed $value, int $min): self
    {
        if (strlen((string)$value) < $min) {
            $this->errors[$field] = "{$field} must be at least {$min}
                characters";
        }
        return $this;
    }

    public function passes(): bool
    {
        return empty($this->errors);
    }

    public function errors(): array
```

```
    {
        return $this->errors;
    }
}
```

1.9 0.9 – Front Controller Bootstrap

`api/index.php` – the single entry point for all API calls:

```php
<?php
declare(strict_types=1);

require_once __DIR__ . '/../vendor/autoload.php';

use App\Config\Environment;
use App\Config\Cors;
use App\Helpers\Response;

// 1. Load environment
Environment::load(__DIR__ . '/config/.env');

// 2. Handle CORS (must be before any output)
Cors::handle();

// 3. Parse route
$method = $_SERVER['REQUEST_METHOD'];
$uri    = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);

// Strip /api prefix (added by Apache Alias)
$uri = preg_replace('#^/api#', '', $uri);
$uri = rtrim($uri, '/') ?: '/';

// 4. Route dispatch (will grow with each phase)
$routes = [];

// Each phase will add routes here:
// $routes['POST']['/auth/login'] = [AuthController::class, 'login'];

// 5. Dispatch
if (isset($routes[$method][$uri])) {
    [$controllerClass, $action] = $routes[$method][$uri];
    $controller = new $controllerClass();
    $controller->$action();
} else {
    Response::error('Route not found', 404);
}
```

[ ] Verify: `http://helix.local/api/nonexistent` returns `{"success":false,"error":"Route not found"}`

1.10 0.10 – Python Environment

[ ] Confirm Python ≥ 3.10: `python -version`

[ ] Create virtual environment inside `python-module/`:

```
cd python-module
python -m venv .venv
```

[ ] Create `pyproject.toml`:

```
[project]
name = "helix-python-module"
version = "0.1.0"
description = "Isolation Forest anomaly detection for HELIX"
requires-python = ">=3.10"

[project.dependencies]
scikit-learn = ">=1.3.0"
pandas = ">=2.0.0"
numpy = ">=1.24.0"
joblib = ">=1.3.0"
mysql-connector-python = ">=8.0.0"

[project.optional-dependencies]
dev = [
    "pytest>=7.0.0",
]

[build-system]
requires = ["setuptools>=68.0"]
build-backend = "setuptools.backends.legacy:BuildBackend"
```

[ ] Install: `pip install -e ".[dev]"` (inside `.venv`)

[ ] Update `PYTHON_PATH` in `.env` to point to `.venv` python binary

1.11 0.11 – Java Environment

[x] Confirm JDK ≥ 17: `java -version`, `javac -version`

[ ] Review `java-module/compile.sh` – understand what it does:

```
#!/bin/bash
mkdir -p build
javac -d build src/LogFormat.java src/ParseResult.java src/LogParser.java
    src/HashScanner.java
echo "Compilation complete"
```

[ ] Run `compile.sh` (or equivalent on Windows with `javac`)

[ ] Verify `.class` files appear in `java-module/build/`

Deliverables

- Git repo with clean structure, `.gitignore`, `CHANGELOG.md`
- Apache vhost serving `frontend/` at root, `api/` at `/api`
- Composer installed with PSR-4 autoloading configured
- `api/config/Environment.php`, `Database.php`, `Cors.php`
- `api/helpers/Response.php`, `Validator.php`
- `api/index.php` Front Controller with basic routing skeleton
- `python-module/pyproject.toml` (replacing `requirements.txt`)
- Java module compiling without errors

Validation Criteria

[ ] `http://helix.local` → serves `frontend/index.html`

[ ] `http://helix.local/api` → JSON response (not HTML)

[ ] `http://helix.local/api/nonexistent` → `{"success":false,"error":"Route not found"}`

[ ] `composer dump-autoload` runs without errors

[ ] Python: `python -c "import sklearn; print('OK')"` inside `.venv`

[ ] Java: `javac` compiles all source files

[ ] Git log shows meaningful commits

Common Mistakes to Avoid

× Skipping the Apache Alias directive – frontend won't be able to call `/api/*`
× Committing `.env` – real credentials must stay out of git
× Keeping `requirements.txt` – you decided to use `pyproject.toml`
× Not setting up PSR-4 autoloading – you'll write manual `require` statements everywhere
× Writing route logic directly in `api/index.php` instead of dispatching to controllers

2. PHASE 1 – Architecture Design, Database Schema & UML

Phase 1: Architecture & Design

Why this comes before coding: You already have `.puml` diagram files. This phase is about validating them, locking the database schema, and mapping every SRS requirement to a concrete implementation path. Changing the schema after writing 5 models is expensive.

Objective

Finalize and validate the complete database schema, UML diagrams, API contract, and traceability matrix. Nothing functional gets built in this phase – only design artifacts.

Learning Goals

- Understand relational database normalization (1NF, 2NF, 3NF)
- Understand how to derive a schema from use cases and actors
- Understand REST API contract-first design
- Understand what each UML diagram type communicates and to whom

Key Theoretical Concepts

- **Entity-Relationship vs Class Diagram:** ER models data; Class models code structure
- **Foreign key CASCADE rules:** `ON DELETE CASCADE` vs `ON DELETE SET NULL` – when to use each
- **ENUM vs lookup table:** ENUMs are fast and sufficient for a PoC; lookup tables scale better
- **REST resource naming:** `/alerts` not `/getAlerts`; `/alerts/{id}/status` not `/updateAlertStatus`
- **HTTP status codes:** 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 500 Internal Server Error

2.1 1.1 – Database Schema

`db/schema.sql` – finalize all tables before writing a single model:

```
CREATE DATABASE IF NOT EXISTS helix_db CHARACTER SET utf8mb4 COLLATE
   utf8mb4_unicode_ci;
USE helix_db;

CREATE TABLE users (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    username      VARCHAR(50)  UNIQUE NOT NULL,
    email         VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role          ENUM('administrator','blue_team','red_team','purple_team
       ','learner') NOT NULL,
    is_active     BOOLEAN      DEFAULT TRUE,
    created_at    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    last_login    TIMESTAMP    NULL
);

CREATE TABLE log_entries (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    filename    VARCHAR(255),
    source_ip   VARCHAR(45),
    timestamp   TIMESTAMP    NOT NULL,
    severity    ENUM('LOW','MEDIUM','HIGH','CRITICAL') DEFAULT 'LOW',
    event_type  VARCHAR(100),
    message     TEXT         NOT NULL,
    raw_line    TEXT,
    ingested_at TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    ingested_by INT          NULL,
    FOREIGN KEY (ingested_by) REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE anomaly_scores (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    log_entry_id  INT   NOT NULL,
    score         FLOAT NOT NULL,
    severity_label ENUM('LOW','MEDIUM','HIGH','CRITICAL') NOT NULL,
    computed_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (log_entry_id) REFERENCES log_entries(id) ON DELETE
       CASCADE
);

CREATE TABLE alerts (
    id               INT AUTO_INCREMENT PRIMARY KEY,
    log_entry_id     INT  NULL,
    anomaly_score_id INT  NULL,
    title            VARCHAR(255) NOT NULL,
    description      TEXT,
    severity         ENUM('LOW','MEDIUM','HIGH','CRITICAL') NOT NULL,
    status           ENUM('open','investigating','resolved') DEFAULT 'open
       ',
    assigned_to      INT  NULL,
```

```
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
       CURRENT_TIMESTAMP,
    FOREIGN KEY (log_entry_id)     REFERENCES log_entries(id)    ON DELETE
        SET NULL,
    FOREIGN KEY (anomaly_score_id) REFERENCES anomaly_scores(id) ON DELETE
        SET NULL,
    FOREIGN KEY (assigned_to)      REFERENCES users(id)          ON DELETE
        SET NULL
);

CREATE TABLE chat_sessions (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    user_id      INT  NOT NULL,
    alert_id     INT  NULL,
    message_role ENUM('user','assistant') NOT NULL,
    content      TEXT NOT NULL,
    sent_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id)  REFERENCES users(id)  ON DELETE CASCADE,
    FOREIGN KEY (alert_id) REFERENCES alerts(id) ON DELETE SET NULL
);

CREATE TABLE scan_results (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    performed_by INT NOT NULL,
    target       VARCHAR(255) NOT NULL,
    port         INT NOT NULL,
    protocol     VARCHAR(10),
    status       ENUM('open','closed','filtered') NOT NULL,
    scanned_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (performed_by) REFERENCES users(id)
);
```

[ ] Verify schema matches your UML diagrams

[ ] Import: `mysql -u helix_user -p helix_db < db/schema.sql`

[ ] Confirm all 6 tables appear in phpMyAdmin

Note on migrations: Every schema change after the initial import must be a new migration file: `002_`, `003_`, etc. Never re-edit `schema.sql` to fix an existing column – add a migration.

2.2 1.2 – API Route Contract

Document every route in `docs/SDD/API_REFERENCE.md`. For each route define: method, path, required role, request body, success response, error responses.

Complete route table:

Method	Path	Controller.Method	Required Role	FR
POST	/auth/register	AuthController.register	public	FR-007
POST	/auth/login	AuthController.login	public	FR-001
POST	/auth/logout	AuthController.logout	any authenticated	FR-005
GET	/auth/me	AuthController.me	any authenticated	FR-003
GET	/dashboard/stats	DashboardController.stats	any authenticated	FR-010
POST	/logs/upload	LogController.upload	admin, blue, purple	FR-020
GET	/logs	LogController.index	any authenticated	FR-022
GET	/logs/{id}	LogController.show	any authenticated	FR-023
GET	/alerts	AlertController.index	any authenticated	FR-030
GET	/alerts/{id}	AlertController.show	any authenticated	FR-034
PUT	/alerts/{id}/status	AlertController.updateStatus	admin, blue, purple	FR-033
POST	/ml/score	MLController.score	administrator	FR-040
GET	/ml/scores	MLController.index	any authenticated	FR-044
POST	/ai/chat	AIController.chat	any authenticated	FR-050
GET	/ai/history	AIController.history	any authenticated	FR-053
POST	/scanner/scan	ScannerController.scan	admin, red, purple	FR-060
GET	/scanner/results	ScannerController.results	admin, red, purple	FR-062

[ ] Add this table to `docs/SDD/API_REFERENCE.md`

[ ] Add every route to your `api/index.php` routing table (as empty stubs for now)

2.3 1.3 – UML Diagram Validation

You already have `.puml` files. This task is about reviewing them for correctness:

[ ] `use_case.puml` – Does it show all 7 actors (A0–A7)? Are `«extend»` / `«include»` used correctly?

[ ] `er_diagram.puml` – Does it match `schema.sql` exactly? Check column names, types, FK arrows

[ ] `class_services.puml` – Does it show Controller → Service → Model relationships?

[ ] `sequence_auth.puml` – Does it show: browser → index.php → AuthMiddleware → AuthController → AuthService → User model → MySQL?

[ ] `sequence_log_ingest.puml` – Does it show the pipeline: upload → LogService → JavaBridge → LogParser → LogEntry model → MLService trigger?

[ ] `sequence_ai_chat.puml` – Does it show: AIController → AIService → OpenAI API → ChatSession model?

[ ] `component.puml` – Does it show: Browser, Apache (frontend + api), PHP (controllers/services/models), Python ML, Java module, MySQL?

[ ] `activity_alert.puml` – Does it show the full alert lifecycle: log ingested → scored → alert created → triaged → resolved?

Export all diagrams as PNG into `docs/UML/png/`.

2.4 1.4 – SRS Traceability Matrix

`docs/traceability_matrix.md`:

FR ID	Description	Route	Controller.Method	DB Table	Status
FR-001	Login	POST /auth/login	AuthController.login	users	[[ ]]
FR-002	Failed login handling	POST /auth/login	AuthController.login	users	[[ ]]
FR-003	Session persistence	GET /auth/me	AuthController.me	users	[[ ]]
		... (map every FR-001 to FR-063)			

[ ] Map every FR-001 to FR-063 to a route, controller method, and DB table

[ ] This matrix is your implementation checklist – check FRs off as you build

2.5 1.5 – Seed File

`database/seed.sql`:

```
-- Admin user (password: Admin@1234)
-- Generate hash: php -r "echo password_hash('Admin@1234', PASSWORD_BCRYPT, ['cost'=>12]);"
INSERT INTO users (username, email, password_hash, role) VALUES
('admin', 'admin@helix.local', '$2y$12$GENERATED_HASH_HERE', 'administrator'),
('blue1', 'blue@helix.local',  '$2y$12$GENERATED_HASH_HERE', 'blue_team'),
('red1',  'red@helix.local',   '$2y$12$GENERATED_HASH_HERE', 'red_team');
```

[ ] Generate real bcrypt hashes and update seed.sql

[ ] Import: `mysql -u helix_user -p helix_db < database/seed.sql`

Deliverables

- `db/schema.sql` – final, importable, 6 tables
- `docs/SDD/API_REFERENCE.md` – complete route contract
- All UML diagrams validated and exported to PNG

- `docs/traceability_matrix.md` – all FRs mapped
- `database/seed.sql` – 3 test users

Validation Criteria

[ ] Schema imports without error

[ ] All 6 tables visible in phpMyAdmin

[ ] All UML diagrams reviewed and corrected

[ ] All FR-001 to FR-063 appear in traceability matrix

[ ] All routes registered as stubs in `api/index.php`

[ ] Git commit: `docs: finalize schema, API contract, UML and traceability matrix`

Common Mistakes to Avoid

× Editing `schema.sql` after first import instead of creating a migration
× Skipping the traceability matrix – you will lose track of which FRs are done
× Routes in `api/index.php` pointing to non-existent controller methods
× UML diagrams that don't match the actual code structure

3. PHASE 2 – Authentication Module (FR-001 to FR-007)

Phase 2: Authentication & RBAC

Why this comes before all other modules: Every other feature is gated behind authentication. You cannot build role-based features without a working session system. Auth is also your highest-security concern – if it's broken, everything built on it is insecure.

Objective

Implement the complete authentication flow: register, login, logout, session validation – with bcrypt hashing, PHP sessions, and role middleware enforcing access control on every protected route.

Learning Goals

- Understand bcrypt and why password hashing ≠ encryption
- Understand PHP session security: `session_regenerate_id()`, cookie flags
- Understand the difference between Authentication (who are you?) and Authorization (what can you do?)
- Understand how middleware chains work in a Front Controller pattern

Key Theoretical Concepts

- **bcrypt cost factor:** Each increment doubles computation time – cost 12 is ~300ms, which is too slow to brute-force
- **Session fixation attack:** Why `session_regenerate_id(true)` is mandatory after login
- **HttpOnly cookie flag:** Prevents JavaScript from reading the session cookie (XSS protection)
- **SameSite=Strict:** Prevents the browser from sending the cookie on cross-site requests (CSRF protection)
- **Generic error messages:** "Invalid credentials" – never "User not found" or "Wrong password"

3.1 2.1 – Auth Middleware

api/middleware/AuthMiddleware.php:

```php
<?php
namespace App\Middleware;

use App\Helpers\Response;

class AuthMiddleware
{
    public static function handle(): void
    {
        self::startSecureSession();

        if (!isset($_SESSION['user_id'])) {
            Response::unauthorized();
        }
    }

    public static function startSecureSession(): void
    {
        if (session_status() === PHP_SESSION_NONE) {
            ini_set('session.cookie_httponly', '1');
            ini_set('session.cookie_samesite', 'Strict');
            ini_set('session.gc_maxlifetime', (string)($_ENV['
                SESSION_LIFETIME'] ?? 3600));
            session_start();
        }
    }

    public static function currentUser(): array
    {
        return [
            'id'   => $_SESSION['user_id']   ?? null,
            'role' => $_SESSION['user_role'] ?? null,
        ];
    }
}
```

api/middleware/RoleMiddleware.php:

```php
<?php
namespace App\Middleware;

use App\Helpers\Response;

class RoleMiddleware
{
    public static function require(array $allowedRoles): void
    {
        AuthMiddleware::handle();

        $userRole = $_SESSION['user_role'] ?? '';
```

```
13
14          if (!in_array($userRole, $allowedRoles, true)) {
15              Response::forbidden();
16          }
17      }
18  }
```

3.2 2.2 – User Model

api/models/User.php:

```
1   <?php
2   namespace App\Models;
3
4   use App\Config\Database;
5   use PDO;
6
7   class User
8   {
9       public static function findByUsername(string $username): ?array
10      {
11          $db   = Database::getInstance();
12          $stmt = $db->prepare('SELECT * FROM users WHERE username = ? AND
                is_active = 1 LIMIT 1');
13          $stmt->execute([$username]);
14          return $stmt->fetch() ?: null;
15      }
16
17      public static function findById(int $id): ?array
18      {
19          $db   = Database::getInstance();
20          $stmt = $db->prepare('SELECT id, username, email, role, created_at
                , last_login FROM users WHERE id = ? LIMIT 1');
21          $stmt->execute([$id]);
22          return $stmt->fetch() ?: null;
23      }
24
25      public static function create(string $username, string $email, string
            $passwordHash, string $role = 'learner'): int
26      {
27          $db   = Database::getInstance();
28          $stmt = $db->prepare('INSERT INTO users (username, email,
                password_hash, role) VALUES (?, ?, ?, ?)');
29          $stmt->execute([$username, $email, $passwordHash, $role]);
30          return (int) $db->lastInsertId();
31      }
32
33      public static function updateLastLogin(int $id): void
34      {
35          $db   = Database::getInstance();
36          $stmt = $db->prepare('UPDATE users SET last_login = NOW() WHERE id
```

```
            = ?');
        $stmt->execute([$id]);
    }

    public static function emailOrUsernameExists(string $username, string $email): bool
    {
        $db   = Database::getInstance();
        $stmt = $db->prepare('SELECT id FROM users WHERE username = ? OR email = ? LIMIT 1');
        $stmt->execute([$username, $email]);
        return (bool) $stmt->fetch();
    }
}
```

3.3 2.3 – Auth Service

api/services/AuthService.php:

```
<?php
namespace App\Services;

use App\Models\User;
use App\Middleware\AuthMiddleware;

class AuthService
{
    public function register(array $data): array
    {
        if (User::emailOrUsernameExists($data['username'], $data['email'])) {
            return ['error' => 'Username or email already exists', 'code' => 409];
        }

        $hash = password_hash($data['password'], PASSWORD_BCRYPT, ['cost' => 12]);
        $id   = User::create($data['username'], $data['email'], $hash);

        return ['user_id' => $id];
    }

    public function login(string $username, string $password): array
    {
        $user = User::findByUsername($username);

        if (!$user || !password_verify($password, $user['password_hash'])) {
            return ['error' => 'Invalid credentials', 'code' => 401];
        }

```

```
        AuthMiddleware::startSecureSession();
        session_regenerate_id(true);

        $_SESSION['user_id']   = $user['id'];
        $_SESSION['user_role'] = $user['role'];

        User::updateLastLogin($user['id']);

        return [
            'user' => [
                'id'       => $user['id'],
                'username' => $user['username'],
                'role'     => $user['role'],
            ]
        ];
    }

    public function logout(): void
    {
        AuthMiddleware::startSecureSession();
        $_SESSION = [];
        session_destroy();
    }

    public function currentUser(): ?array
    {
        $id = $_SESSION['user_id'] ?? null;
        return $id ? User::findById((int)$id) : null;
    }
}
```

3.4 2.4 – Auth Controller

api/controllers/AuthController.php:

```
<?php
namespace App\Controllers;

use App\Services\AuthService;
use App\Middleware\AuthMiddleware;
use App\Helpers\Response;
use App\Helpers\Validator;

class AuthController
{
    private AuthService $service;

    public function __construct()
    {
        $this->service = new AuthService();
    }
```

```
17
18      public function register(): void
19      {
20          $body = json_decode(file_get_contents('php://input'), true) ?? [];
21
22          $v = (new Validator())
23              ->required('username', $body['username'] ?? '')
24              ->required('email', $body['email'] ?? '')
25              ->required('password', $body['password'] ?? '')
26              ->email('email', $body['email'] ?? '')
27              ->minLength('password', $body['password'] ?? '', 8);
28
29          if (!$v->passes()) {
30              Response::error(json_encode($v->errors()), 422);
31          }
32
33          $result = $this->service->register($body);
34
35          if (isset($result['error'])) {
36              Response::error($result['error'], $result['code']);
37          }
38
39          Response::success($result, 'Account created', 201);
40      }
41
42      public function login(): void
43      {
44          $body = json_decode(file_get_contents('php://input'), true) ?? [];
45
46          $result = $this->service->login(
47              $body['username'] ?? '',
48              $body['password'] ?? ''
49          );
50
51          if (isset($result['error'])) {
52              Response::error($result['error'], $result['code']);
53          }
54
55          Response::success($result['user'], 'Login successful');
56      }
57
58      public function logout(): void
59      {
60          AuthMiddleware::handle();
61          $this->service->logout();
62          Response::success(null, 'Logged out');
63      }
64
65      public function me(): void
66      {
67          AuthMiddleware::handle();
68          $user = $this->service->currentUser();
69
```

```
        if (!$user) {
            Response::unauthorized();
        }

        Response::success($user);
    }
}
```

3.5 2.5 – Register Auth Routes

In `api/index.php`, add to the routes array:

```
use App\Controllers\AuthController;

$routes['POST']['/auth/register'] = [AuthController::class, 'register'];
$routes['POST']['/auth/login']    = [AuthController::class, 'login'];
$routes['POST']['/auth/logout']   = [AuthController::class, 'logout'];
$routes['GET']['/auth/me']        = [AuthController::class, 'me'];
```

3.6 2.6 – Frontend: Login Page

`frontend/login.html` – already exists in your tree. Build it now:

- HTML form: username + password fields
- No page refresh – use JavaScript `fetch()`

`frontend/js/auth.js`:

```
async function login(username, password) {
  const res = await fetch("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    credentials: "include",
    body: JSON.stringify({ username, password }),
  });

  const data = await res.json();

  if (data.success) {
    sessionStorage.setItem("userRole", data.data.role);
    window.location.href = "/dashboard.html";
  } else {
    showError(data.error);
  }
}
```

`frontend/js/api.js` – the shared API client module:

```
const API_BASE = "/api";
```

```
async function apiFetch(path, options = {}) {
  const defaults = {
    credentials: "include",
    headers: { "Content-Type": "application/json" },
  };

  const res = await fetch(`${API_BASE}${path}`, { ...defaults, ...options
     });

  if (res.status === 401) {
    window.location.href = "/login.html";
    return;
  }

  return await res.json();
}

const api = {
  get: (path) => apiFetch(path),
  post: (path, body) =>
    apiFetch(path, { method: "POST", body: JSON.stringify(body) }),
  put: (path, body) =>
    apiFetch(path, { method: "PUT", body: JSON.stringify(body) }),
  delete: (path) => apiFetch(path, { method: "DELETE" }),
};
```

Why `credentials: 'include'`: Without this, the browser won't send the session cookie with cross-path requests. This is the most common cause of "keeps logging me out" bugs.

3.7 2.7 – Auth Tests

`tests/php/AuthServiceTest.php` (PHPUnit):

```
<?php
namespace Tests;

use PHPUnit\Framework\TestCase;
use App\Services\AuthService;

class AuthServiceTest extends TestCase
{
    public function test_register_returns_user_id(): void
    {
        $this->markTestIncomplete('Requires test DB setup');
    }

    public function test_login_returns_error_on_wrong_password(): void
    {
        $service = new AuthService();
        $result  = $this->invokeMethod($service, 'validateCredentials', ['
```

```
            nonexistent', 'wrongpass']);
        $this->assertArrayHasKey('error', $result);
    }

    private function invokeMethod(object $obj, string $method, array $args
        = []): mixed
    {
        $ref = new \ReflectionMethod($obj, $method);
        $ref->setAccessible(true);
        return $ref->invokeArgs($obj, $args);
    }
}
```

3.8 2.8 – Manual Test Checklist

`tests/manual/auth_tests.md`:

[ ] POST `/api/auth/register` valid data → 201, user_id returned

[ ] POST `/api/auth/register` duplicate username → 409

[ ] POST `/api/auth/register` invalid email → 422

[ ] POST `/api/auth/login` valid credentials → 200, role returned, cookie set

[ ] POST `/api/auth/login` wrong password → 401, "Invalid credentials"

[ ] GET `/api/auth/me` with cookie → 200, user data

[ ] GET `/api/auth/me` without cookie → 401

[ ] POST `/api/auth/logout` → 200, cookie destroyed

[ ] GET `/api/auth/me` after logout → 401

[ ] Test route requiring admin role as learner → 403

Test all using Postman or Insomnia – configure "Send cookies" option.

Deliverables

- `api/middleware/AuthMiddleware.php`, `RoleMiddleware.php`
- `api/models/User.php`
- `api/services/AuthService.php`
- `api/controllers/AuthController.php`
- `frontend/login.html` functional form
- `frontend/js/auth.js`, `frontend/js/api.js`

- Auth routes registered in `api/index.php`
- `tests/manual/auth_tests.md` all checks passing

Validation Criteria

[ ] FR-001 to FR-007 checked in traceability matrix

[ ] Passwords stored as bcrypt (verify in phpMyAdmin – hash starts with `$2y$`)

[ ] Wrong password returns same error message as non-existent user

[ ] `session_regenerate_id(true)` called on every login

[ ] Frontend redirects to login on 401 (handled in `api.js`)

[ ] Git commit: `feat(auth): implement authentication, RBAC middleware, login frontend`

Common Mistakes to Avoid

× Checking roles only in the frontend JavaScript – always enforce in `RoleMiddleware`
× Returning "User not found" vs "Wrong password" separately – merge into one message
× Forgetting `credentials: 'include'` in every `fetch()` call
× Using `md5()` or `sha1()` for passwords
× Not calling `session_regenerate_id()` – session fixation vulnerability

4. PHASE 3 – Dashboard Module (FR-010 to FR-014)

Phase 3: Dashboard & Routing

Why now: The dashboard is the shell of the application – the first page after login. Building it now, even with placeholder data, establishes the UI framework, client-side router, and nav structure that all subsequent modules will plug into.

Objective

Build the dashboard frontend shell (with routing), implement the stats API endpoint, and render security metrics with Chart.js. Use placeholder/zero data for sections not yet implemented.

Learning Goals

- Understand client-side routing without a framework (`router.js`)
- Understand the HTML5 History API (`pushState`, `popstate`)
- Understand Chart.js configuration and data binding
- Understand role-based UI rendering in JavaScript

Key Theoretical Concepts

- **SPA routing:** The browser loads one HTML page; JavaScript swaps content based on URL without full page reload
- `history.pushState()`: Changes the URL without navigating – the foundation of client-side routing
- **Dashboard as aggregation endpoint:** The dashboard API doesn't own data; it queries multiple tables and returns a summary
- **Graceful empty states:** Your dashboard must not crash when the DB is empty (it will be during early development)

4.1 3.1 – Client-Side Router

frontend/js/router.js:

```
const routes = {
  "/dashboard": () => import("./dashboard.js").then((m) => m.render()),
  "/logs": () => import("./logs.js").then((m) => m.render()),
  "/alerts": () => import("./alerts.js").then((m) => m.render()),
  "/ai": () => import("./chat.js").then((m) => m.render()),
  "/scanner": () => import("./scanner.js").then((m) => m.render()),
};

async function navigate(path) {
  const handler = routes[path];

  if (!handler) {
    document.getElementById("app").innerHTML = "<h2>404 - Page not found</
       h2>";
    return;
  }

  history.pushState({}, "", path);
  document.getElementById("app").innerHTML =
    '<div class="loading">Loading...</div>';
  await handler();
}

window.addEventListener("popstate", () => navigate(location.pathname));

document.addEventListener("click", (e) => {
  const link = e.target.closest("[data-route]");
  if (link) {
    e.preventDefault();
    navigate(link.dataset.route);
  }
});

export { navigate };
```

4.2 3.2 – Dashboard Shell

frontend/dashboard.html:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>HELIX - Dashboard</title>
    <link rel="stylesheet" href="/css/main.css" />
    <link rel="stylesheet" href="/css/dashboard.css" />
  </head>
  <body>
```

```
    <div id="layout">
      <nav id="sidebar"></nav>
      <main id="app"></main>
    </div>
    <script type="module" src="/js/ui.js"></script>
    <script type="module" src="/js/router.js"></script>
  </body>
</html>
```

`frontend/js/ui.js` – builds the sidebar based on user role:

```
import { navigate } from "./router.js";

const roleNavMap = {
  administrator: ["/dashboard", "/logs", "/alerts", "/ai", "/scanner"],
  blue_team: ["/dashboard", "/logs", "/alerts", "/ai"],
  red_team: ["/dashboard", "/scanner", "/ai"],
  purple_team: ["/dashboard", "/logs", "/alerts", "/ai", "/scanner"],
  learner: ["/dashboard", "/ai"],
};

async function initUI() {
  const res = await fetch("/api/auth/me", { credentials: "include" });
  if (!res.ok) {
    window.location.href = "/login.html";
    return;
  }

  const { data: user } = await res.json();
  const allowedPaths = roleNavMap[user.role] ?? [];

  buildSidebar(allowedPaths, user);
  navigate("/dashboard");
}

function buildSidebar(paths, user) {
  const labels = {
    "/dashboard": "Dashboard",
    "/logs": "Logs",
    "/alerts": "Alerts",
    "/ai": "AI Assistant",
    "/scanner": "Scanner",
  };
  const nav = document.getElementById("sidebar");
  nav.innerHTML =
    paths
      .map((p) => `<a data-route="${p}" href="${p}">${labels[p]}</a>`)
      .join("") + `<button id="logout-btn">Logout (${user.username})</
         button>`;

  document.getElementById("logout-btn").addEventListener("click", async ()
      => {
    await fetch("/api/auth/logout", { method: "POST", credentials: "
```

```
        include" });
    window.location.href = "/login.html";
  });
}

initUI();
```

4.3 3.3 – Dashboard Stats Endpoint

api/controllers/DashboardController.php:

```
<?php
namespace App\Controllers;

use App\Config\Database;
use App\Middleware\AuthMiddleware;
use App\Helpers\Response;

class DashboardController
{
    public function stats(): void
    {
        AuthMiddleware::handle();
        $db = Database::getInstance();

        $alertsBySeverity = $db->query(
            "SELECT severity, COUNT(*) as count FROM alerts GROUP BY
               severity"
        )->fetchAll();

        $logCount = $db->query("SELECT COUNT(*) as count FROM log_entries"
           )->fetch()['count'];

        $avgScore = $db->query("SELECT ROUND(AVG(score), 4) as avg FROM
           anomaly_scores")->fetch()['avg'] ?? 0;

        $trend = $db->query(
            "SELECT DATE(created_at) as date, COUNT(*) as count
             FROM alerts
             WHERE created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)
             GROUP BY DATE(created_at)
             ORDER BY date ASC"
        )->fetchAll();

        $openAlerts = $db->query(
            "SELECT COUNT(*) as count FROM alerts WHERE status = 'open'"
        )->fetch()['count'];

        Response::success([
            'alerts_by_severity' => $alertsBySeverity,
            'log_count'          => (int) $logCount,
```

```
            'avg_anomaly_score'  => (float) $avgScore,
            'alert_trend'        => $trend,
            'open_alerts'        => (int) $openAlerts,
        ]);
    }
}
```

Register route in `api/index.php`:

```
$routes['GET']['/dashboard/stats'] = [DashboardController::class, 'stats'];
```

4.4 3.4 – Dashboard Frontend Module

`frontend/js/dashboard.js`:

```
import { api } from "./api.js";

export async function render() {
  const { data } = await api.get("/dashboard/stats");

  document.getElementById("app").innerHTML = `
    <div class="dashboard-grid">
      <div class="metric-card">
        <span class="metric-label">Open Alerts</span>
        <span class="metric-value">${data.open_alerts}</span>
      </div>
      <div class="metric-card">
        <span class="metric-label">Total Logs</span>
        <span class="metric-value">${data.log_count}</span>
      </div>
      <div class="metric-card">
        <span class="metric-label">Avg Anomaly Score</span>
        <span class="metric-value">${data.avg_anomaly_score}</span>
      </div>
      <canvas id="severity-chart"></canvas>
      <canvas id="trend-chart"></canvas>
    </div>
  `;

  renderSeverityChart(data.alerts_by_severity);
  renderTrendChart(data.alert_trend);
}

function renderSeverityChart(alerts) {
  // Chart.js doughnut chart
  const ctx = document.getElementById("severity-chart").getContext("2d");
  new Chart(ctx, {
    type: "doughnut",
    data: {
      labels: alerts.map((a) => a.severity),
```

```
      datasets: [{
        data: alerts.map((a) => a.count),
        backgroundColor: ["#dc2626", "#f59e0b", "#3b82f6", "#6b7280"],
      }],
    },
    options: { plugins: { title: { display: true, text: "Alerts by
       Severity" } } },
  });
}

function renderTrendChart(trend) {
  const ctx = document.getElementById("trend-chart").getContext("2d");
  new Chart(ctx, {
    type: "line",
    data: {
      labels: trend.map((t) => t.date),
      datasets: [{
        label: "Alerts per day",
        data: trend.map((t) => t.count),
        borderColor: "#3C3489",
        tension: 0.3,
      }],
    },
    options: { plugins: { title: { display: true, text: "7-Day Alert Trend
       " } } },
  });
}
```

Deliverables

- `frontend/js/router.js`, `frontend/js/ui.js`
- `frontend/dashboard.html` with sidebar and `#app` container
- `frontend/js/dashboard.js` with Chart.js rendering
- `api/controllers/DashboardController.php`
- Dashboard route registered in `api/index.php`
- Chart.js included via CDN in `dashboard.html`

Validation Criteria

- [ ] FR-010 to FR-014 checked in traceability matrix
- [ ] Sidebar shows only role-appropriate links
- [ ] Dashboard renders with zero data (no crashes)

[ ] Chart.js charts render with empty/zero datasets

[ ] Client-side routing works (back/forward buttons)

[ ] Git commit: `feat(dashboard): implement dashboard shell, stats API, Chart.js`

Common Mistakes to Avoid

× Hardcoding role-based nav in HTML – use `ui.js` to build dynamically
× Dashboard crashing on empty DB – always use `?? 0` or `|| []`
× Forgetting `history.pushState()` – browser back button breaks
× Mixing frontend and backend routing – keep them separate

5. PHASE 4 through PHASE 10 – Implementation Phases

Phases 4 through 10 follow the same structural pattern as Phases 0–3:

- **Phase 4:** Log Ingestion Module (FR-020 to FR-023)
- **Phase 5:** Alert Module (FR-030 to FR-034)
- **Phase 6:** ML Detection Module (FR-040 to FR-044)
- **Phase 7:** AI Assistant Module (FR-050 to FR-053)
- **Phase 8:** Scanner Module (FR-060 to FR-062)
- **Phase 9:** Integration & Testing
- **Phase 10:** Docker & Deployment

Each phase includes: Objective, Learning Goals, Key Theoretical Concepts, Practical Tasks with code, Deliverables, Validation Criteria, and Common Mistakes to Avoid.

Conclusion

This roadmap provides a phased, dependency-ordered execution plan for building the HELIX PoC. Each phase is self-contained with clear deliverables, validation criteria, and common pitfalls to avoid. Follow the strict dependency chain – do not skip ahead. Build one module at a time, test it thoroughly, then proceed.